

Airline Reservation System

Project Report & Software Architecture

Author: Rahim Rezaie | Repository: HaiderIkramMaan/AirlineReservation_System
Technology Stack: Java 25 (OpenJDK), JavaFX 21, JSON File Persistence

1. Executive Summary

The **Airline Reservation System** is a desktop application designed to manage airline operations, flight scheduling, user identity management, seat reservations, ticket issuance, payment processing, and administrative analytics. Built using Java 25 and JavaFX 21, the system demonstrates modern object-oriented software engineering principles including **Abstraction**, **Inheritance**, **Polymorphism**, **Encapsulation**, and established design patterns (**Singleton**, **Model-View-Controller**, **Data Serialization**).

2. System Requirements & Functional Scope

Category 1: Authentication, User Management & Notifications

- **Identity Hierarchy:** Abstract Person base class extended by Passenger and Admin.
- **Authentication Service:** AuthenticationService singleton managing user registration, credential verification via checkPassword(), and active session tracking.
- **Notification System:** Notification class dispatching alerts (BOOKING_CONFIRMATION, FLIGHT_UPDATE, CANCELLATION) linked to recipient Person instances.

Category 2: Flight, Seat & Booking Core

- **Flight Inventory:** Flight, Airline, Airport, and FlightRegistry managing flight schedules, seat maps, origins, and destinations.
- **Polymorphic Seat Hierarchy:** Abstract Seat class extended by EconomySeat (1.0x), BusinessSeat (1.5x), and FirstClassSeat (2.5x), providing dynamic pricing calculations and visual layout color rendering.
- **Booking & E-Tickets:** Booking state engine managing statuses (PENDING, CONFIRMED, CANCELLED) and Ticket issuance.

Category 3: Payments, Reports & Dashboards

- **Polymorphic Payments:** Abstract Payment base class with concrete subclasses CreditCardPayment, CashPayment, and BankTransferPayment.
- **Administrative Analytics:** Abstract Report base class extended by RevenueReport, OccupancyReport, and FlightScheduleReport.
- **JavaFX Desktop GUI:** Graphical views (LoginView, RegisterView, PassengerDashboardView, AdminDashboardView) powered by JavaFX.

3. Object-Oriented Software Design & Architecture

- **Abstraction:** Base classes declare abstract contracts without locking concrete subclasses to specific implementations.
- **Polymorphism:** Seat.getPrice() calculates class-specific prices dynamically. Payment.processPayment() executes distinct validation workflows. Report.generate() computes analytical metrics.

- **Encapsulation:** Strict use of private/protected member fields with public accessors/mutators and unmodifiable list views.
- **Interfaces & Serialization:** Generic `DataSerializer<T>` interface establishing standardized JSON serialization contracts.
- **Design Patterns:** Singleton Pattern applied in `AuthenticationService` and `FlightRegistry`. MVC Pattern decoupling domain models, views, and controllers.

4. Core Component & Entity Summary

Package	Core Classes	OOP Principles / Patterns
person	Person, Passenger, Admin, DataSerializer	Inheritance, Polymorphism, Serialization
service	AuthenticationService	Singleton Pattern, Thread-safe Session Map
flight	Flight, Airline, Airport, FlightRegistry, Ticket	Composition, Aggregation, Singleton
seat	Seat, EconomySeat, BusinessSeat, FirstClassSeat	Polymorphism, Abstract Layout Rendering
booking	Booking, BookingStatus	State Machine, Composition
payment	Payment, CreditCardPayment, CashPayment, BankTransferPayment	Polymorphic Payment Validation
report	Report, RevenueReport, OccupancyReport, FlightScheduleReport	Polymorphic Analytics & File Export
ui / controller	LoginView, PassengerDashboardView, AdminDashboardView	JavaFX GUI, MVC Pattern

5. File Persistence & Data Schema

Data persistence is managed by `FileManager` saving structured JSON and text files to the `data/` directory:

- **users.json:** Stores user profiles (Passenger/Admin) serialized via `serializeData()`.
- **flights.json:** Stores flight schedules, airline assignments, origin/destination airports, and seat configurations.
- **bookings.txt:** Persists passenger booking records and statuses.

6. Conclusion

The Airline Reservation System successfully delivers a robust, modular, and maintainable software architecture. By leveraging OOP design patterns and JavaFX, the application provides an extensible codebase ready for future database integration and real-world deployment.